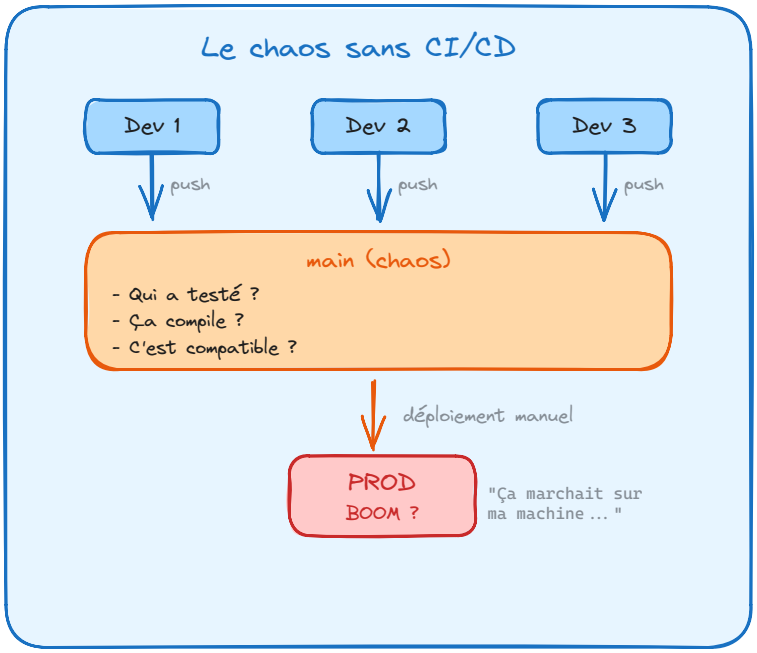
Pipelines CI/CD : automatiser la livraison logicielle

Resoudre ce cauchemar sans fin

Imaginez : vous êtes développeur dans une équipe de 5 personnes. Chaque semaine, c'est le même scénario. Quelqu'un pousse du code, ça casse la production. Personne ne sait exactement ce qui a changé. Les mises en production sont stressantes, souvent le vendredi soir. Et quand ça plante, c'est la panique pour revenir en arrière.

Ce chaos a un nom : **l'absence d'automatisation fiable**.



Une **pipeline CI/CD** résout ce problème. Elle automatise le chemin entre un commit et la production : compilation, tests, validation, déploiement. Plus de "ça marchait sur ma machine". Plus de déploiements stressants. Un feedback immédiat sur chaque modification.

Objectif de ce parcours

À la fin de ce parcours, vous saurez :

1. **Comprendre** ce qu'est une pipeline et pourquoi elle change tout
2. **Distinguer** les différents niveaux d'automatisation (CI, Delivery, Deployment)
3. **Concevoir** une pipeline adaptée à votre contexte
4. **Éviter** les pièges classiques qui ruinent l'efficacité
5. **Sécuriser** votre chaîne pour éviter les attaques

Ce n'est qu'après avoir maîtrisé ces fondamentaux que nous aborderons les outils concrets.

À qui s'adresse ce parcours ?

Vous êtes au bon endroit si vous êtes :

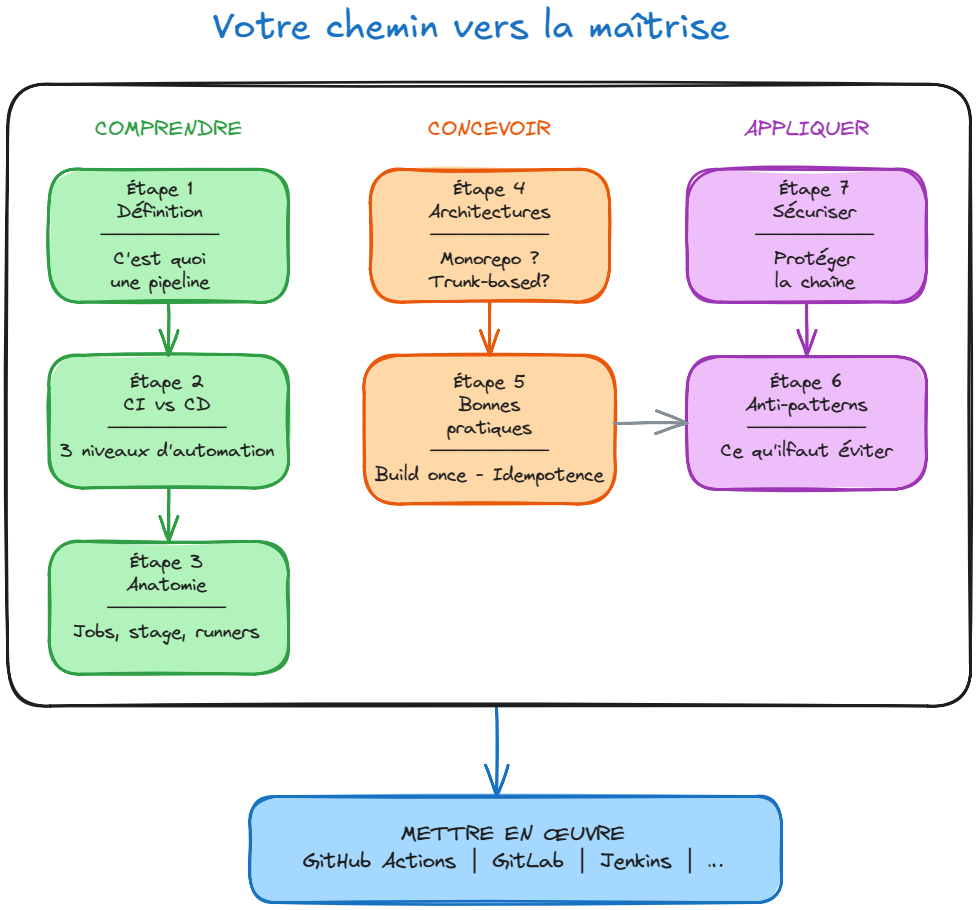
- Développeur souhaitant automatiser vos tests et déploiements
- Ops voulant fiabiliser les mises en production
- Tech Lead cherchant à structurer les pratiques de votre équipe
- Débutant curieux de comprendre le **DevOps** par la pratique

Prérequis :

- Savoir utiliser **Git** (commits, branches, push)
- Être à l'aise avec la ligne de commande **Linux**
- Avoir des notions de **Docker** (recommandé, pas obligatoire)

Votre parcours d'apprentissage

Ce parcours est **progressif**. Chaque étape s'appuie sur la précédente. Résistez à la tentation de sauter directement aux outils : les concepts sont la clé pour faire les bons choix.



Étape 1 — Qu'est-ce qu'une pipeline CI/CD ?

Avant de construire une pipeline, il faut comprendre ce qui la distingue d'un simple script.

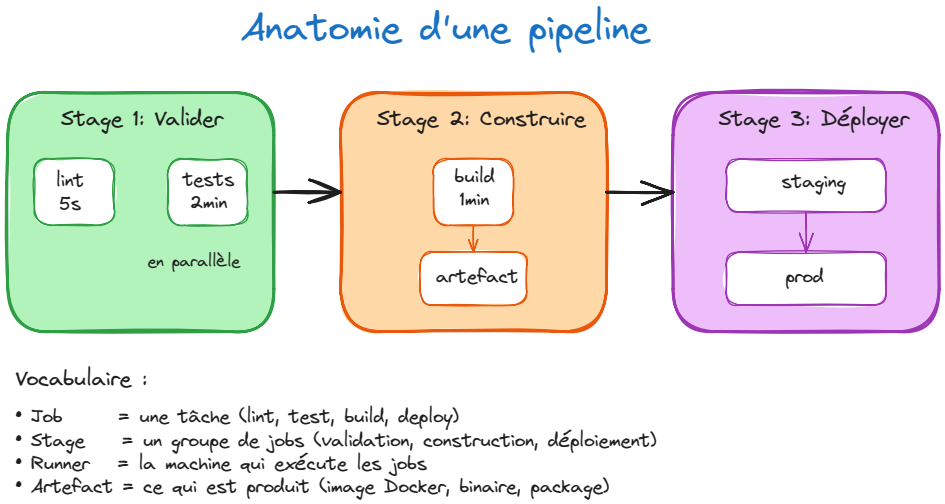
Un script **Bash**, c'est pratique. Vous l'écrivez, vous le lancez, il fait le job. Mais il a des limites :

- Il s'exécute sur **votre** machine (avec **vos** configurations)

- Il ne garde aucune trace de ce qui s'est passé
- Il faut le lancer manuellement
- S'il échoue, vous avez juste "erreur"

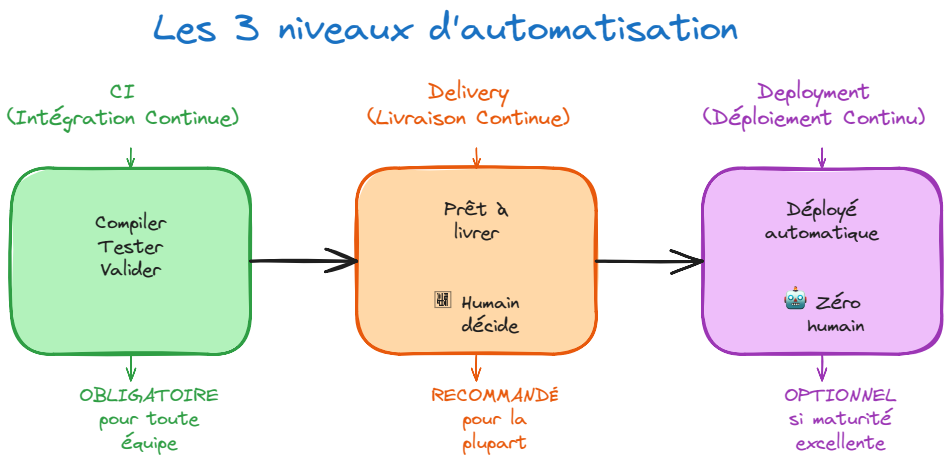
Une pipeline, c'est autre chose. C'est un **système d'exécution gouverné** :

Ce que fait un script	Ce que fait une pipeline
S'exécute sur votre machine	S'exécute sur un serveur dédié, environnement contrôlé
Succès ou échec, point final	Feedback détaillé par étape : lint ✓, tests ×, build —
Aucun historique	Historique complet : qui, quand, quoi, résultat
Déclenché manuellement	Déclenché automatiquement (push, pull request, tag)



Étape 2 — CI, Delivery, Deployment : trois niveaux d'automatisation

Ces trois termes sont constamment confondus. Ils représentent pourtant des niveaux d'automatisation très différents.



Continuous Integration (CI) — Le minimum vital. Chaque push déclenche compilation, tests, analyse. Le feedback arrive en minutes. Si ça casse, vous le savez immédiatement.

Continuous Delivery — Le code est toujours prêt à partir en production. Mais un humain appuie sur le bouton. Idéal quand vous avez des contraintes réglementaires ou une équipe qui monte en compétence.

Continuous Deployment — Chaque modification validée part automatiquement en production. Zéro intervention humaine. Ça exige une couverture de tests excellente et des mécanismes de rollback solides.

Étape 3 — Anatomie d'une pipeline

Maintenant que vous savez ce qu'est une pipeline et ses niveaux d'automatisation, voyons comment elle est structurée concrètement.

Une pipeline se décompose en **stages** (phases) qui contiennent des **jobs** (tâches). Les stages s'exécutent en séquence, les jobs d'un même stage peuvent s'exécuter en parallèle.

Étape 4 — Choisir la bonne architecture

Toutes les pipelines ne se ressemblent pas. Votre architecture dépend de votre contexte : taille de l'équipe, organisation du code, fréquence de déploiement.